



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Python para Ciencia de Datos

Tema 9 **Arrays de NumPy** **para Ciencia de Datos**

Máster F.P. En IA & Big Data ANALYTICS
Curso 2026-2027

UPV

Índice



- 1. Ventajas de NumPy**
- 2. NumPy básico**
- 3. Operadores**
- 4. Indexing y Slicing**
- 5. Transposiciones**

Resultados de aprendizaje

- Comprender el uso de NumPy para ser capaz de manejar datos eficientemente
- Comprender los conceptos básicos de NumPy, indexado, slicing y cómo aplicarlos
- Operar de forma óptima con arrays de NumPy
- Elegir la estructura de datos más adecuada para cada problema

Ventajas de NumPy

Concepto

- NumPy es la **base** del **ecosistema científico** de Python
- Python puro es **flexible** pero **lento** para cálculo numérico.
- Grandes volúmenes de datos (Big Data, IA, ML) requieren **eficiencia**.
- Necesitamos:
 - Computación **vectorizada**
 - Operaciones eficientes en **memoria**
 - Integración con **librerías científicas**

Ventajas de NumPy

⚡ Velocidad

- Arrays de NumPy están implementados en **C** → mucho más rápidos que las listas de Python.
- Evita bucles explícitos en Python → operaciones **vectorizadas**.

```
[1]: import numpy as np
import time

a = np.arange(1e6)
b = list(range(int(1e6)))

%timeit a*2
%timeit [x*2 for x in b]
```

195 µs ± 9.2 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
30.4 ms ± 2.67 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)

👉 Diferencias de órdenes de magnitud.

Velocidad

Blue Yellow Red



a[0]	a[1]	a[2]
Blue	Yellow	Red



Ventajas de NumPy

⚡ Velocidad

Acceso aleatorio

Lectura rápida – Escritura lenta



Green Escribir en segunda posición



Green



Ventajas de NumPy

⚡ Velocidad

Blue Yellow Red

Acceso aleatorio

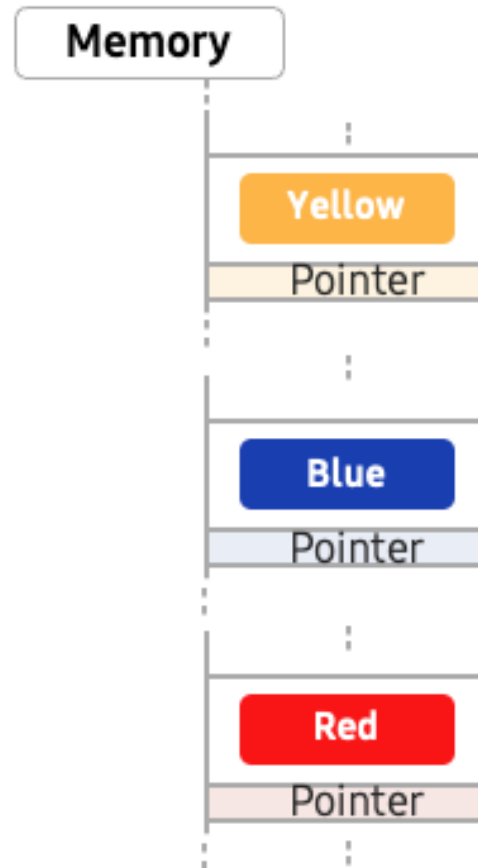
Eliminar la segunda posición



Ventajas de NumPy

⚡ Velocidad

Acceso enlazado

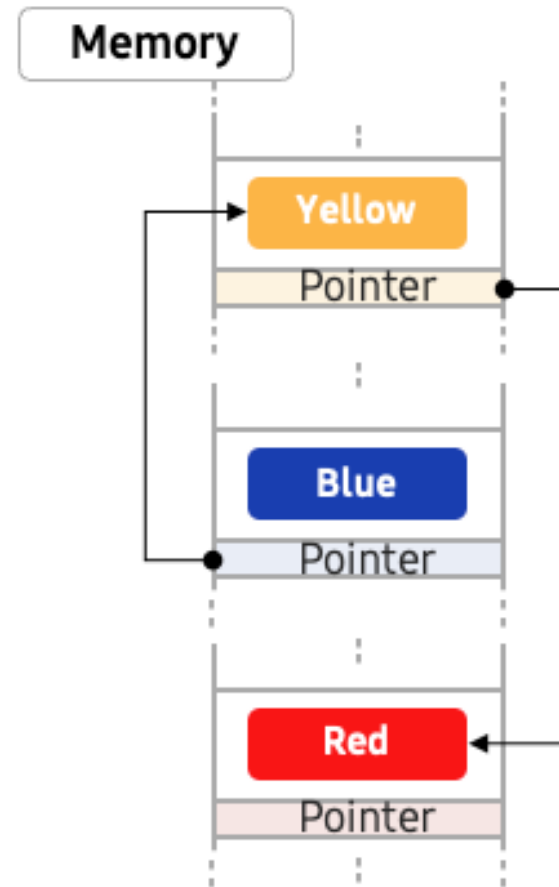


Ventajas de NumPy

⚡ Velocidad



Acceso enlazado



Ventajas de NumPy

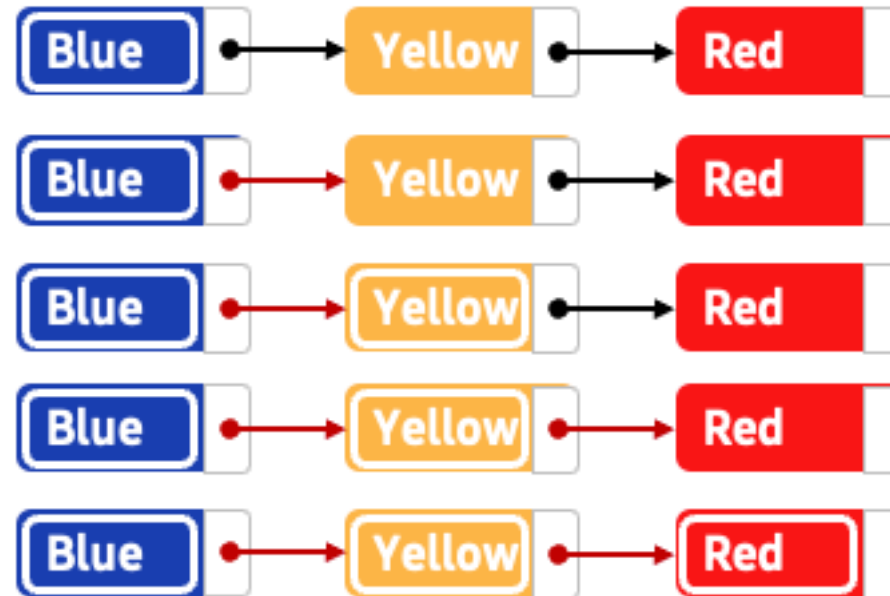
⚡ Velocidad



Acceso enlazado

Lectura lenta – Escritura rápida

Sequential Access

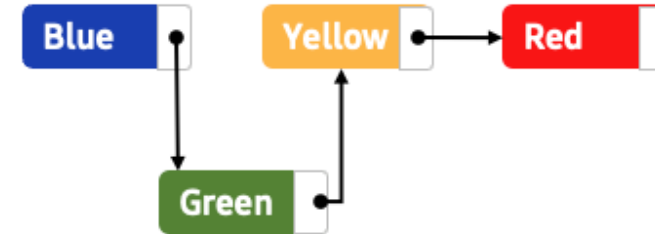
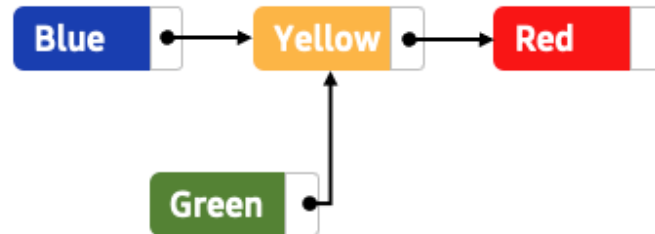


Ventajas de NumPy

⚡ Velocidad

Acceso enlazado

Lectura lenta – Escritura rápida



Ventajas de NumPy

Eficiencia en memoria

-  Menor consumo de **memoria**
 - Listas de Python guardan **referencias a objetos** → overhead alto.
 - NumPy almacena datos en **bloques contiguos** de memoria (**C arrays**).
-  Ejemplo:
 - Lista de 1 millón de enteros $\approx$ 40 MB
 - NumPy array de 1 millón de enteros $\approx$ 8 MB

Ventajas de NumPy

Funcionalidad avanzada

🔧 NumPy incluye:

- Objetos array y matrices **n-dimensionales** de tipo de datos **homogéneo**
- Operaciones **vectorizadas** y **matriciales**
- Funciones **matemáticas** de alto nivel (trigonometría, álgebra lineal, estadísticas)
- **Broadcasting** (aplicar operaciones en arrays de distintas dimensiones)
- **Indexación** avanzada y **slicing**

👉 Permite construir desde modelos de **IA** básicos hasta **redes neuronales**.

Ventajas de NumPy

Ecosistema



NumPy es la base de:

- Pandas (análisis de datos)
- Scikit-learn (machine learning)
- SciPy (cálculo científico)
- TensorFlow, PyTorch, JAX (deep learning, GPU computing)

NumPy Básico

Características

- Los arrays de NumPy tienen un **tamaño fijo** en memoria (a diferencia de las listas de Python)
- **Cambiar** el tamaño de un NumPy array implica crear uno **nuevo**
- Los elementos de un array son del mismo **tipo** y ocupan la misma cantidad de **memoria**
- Sus operaciones **minimizan** el uso de **bucles** mediante la **vectorización** (usan operaciones internas en C más rápidas).

NumPy Básico

Como importar NumPy

```
[1]: import numpy as np
```

```
[2]: np.__version__
```

```
[2]: '2.1.0'
```

- La convención es usar la abreviatura `np` al importar
- Usaremos la versión 2 (aunque la 1 es todavía muy utilizada)

NumPy Básico

Crear un array de NumPy

```
[3]: np.array([1,3,5,7])
```

```
[3]: array([1, 3, 5, 7])
```

```
[4]: arr1 = np.array([1,3,5,7])
```

```
[5]: type(arr1)
```

```
[5]: numpy.ndarray
```

- Un array de NumPy se crea a partir de cualquier iterable
- Su tipo es `numpy.ndarray` (array n-dimensional)

NumPy Básico

Crear un array de NumPy

```
[6]: arr2 = np.array([1,3,5,7])
```

```
[7]: id(arr1)
```

```
[7]: 4544511120
```

```
[8]: id(arr2)
```

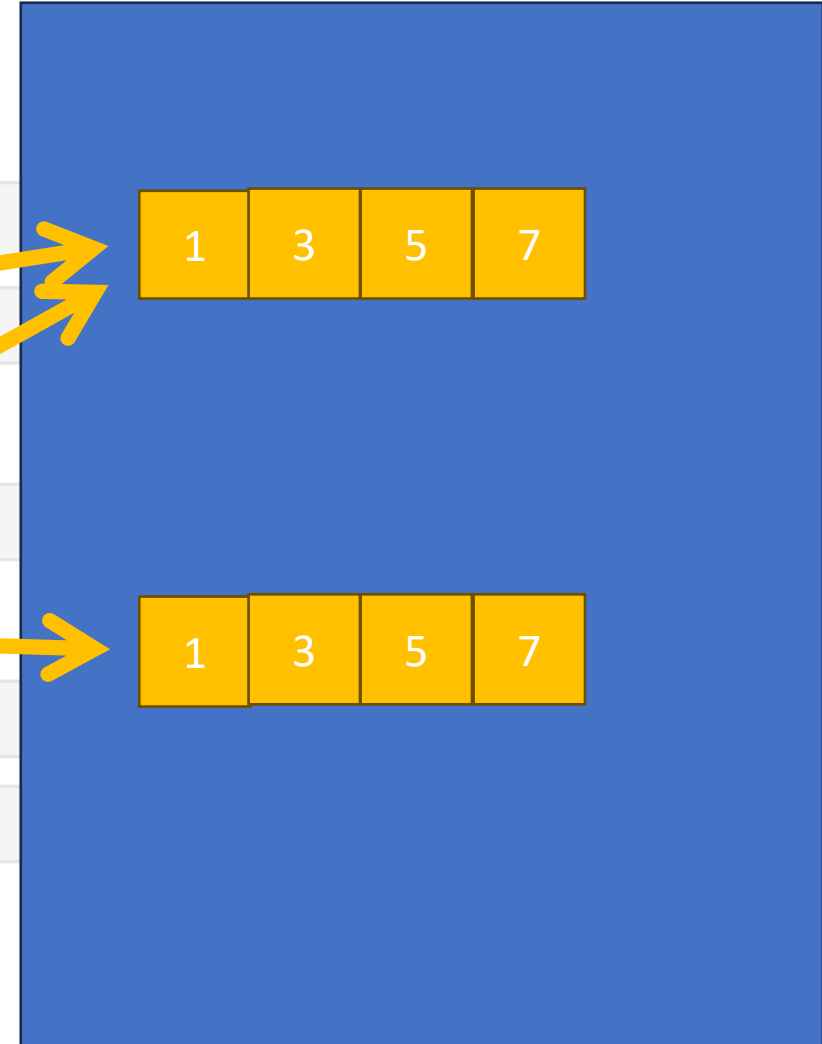
```
[8]: 4544510832
```

```
[9]: arr3=arr1
```

```
[10]: id(arr3)
```

```
[10]: 4544511120
```

- Cada array es único
- El operador de asignación copia la referencia



NumPy Básico

Crear un array de NumPy

```
[12]: np.array((1,3,5,7,9))
```

```
[12]: array([1, 3, 5, 7, 9])
```

```
[13]: np.array({1,2,2,2,2,3,3,3,4,4,4,4,4,5})
```

```
[13]: array({1, 2, 3, 4, 5}, dtype=object)
```

- Se pueden crear arrays a partir de tuplas o sets
- Aunque lo más habitual es a partir de listas

NumPy Básico

Crear un array de NumPy

```
[14]: np.arange(10)
```

```
[14]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
[15]: np.arange(1, 10)
```

```
[15]: array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
[16]: np.arange(1, 10, 2)
```

```
[16]: array([1, 3, 5, 7, 9])
```

- NumPy proporciona su propio generador para rangos (`np.arange`)
- Utiliza los mismos parámetros que el `range` de Python (`start`, `end`, `step`)

NumPy Básico

Crear un array de NumPy

```
[17]: len10 = np.arange(10)
```

```
[18]: len(len10)
```

```
[18]: 10
```

```
[19]: len10.size
```

```
[19]: 10
```

- El tamaño de un array de NumPy se puede consultar con la función `len` de Python o con su propiedad `size`.

NumPy Básico

Crear un array de NumPy

```
[20]: arr1 = np.array([1,2,3,4])  
      arr1.dtype
```

```
[20]: dtype('int64')    NumPy trata de detectar el tipo de datos por defecto
```

```
[21]: arr2 = np.array([1.0, 2.0, 3.0, 4.0])  
      arr2.dtype
```

```
[21]: dtype('float64')
```

```
[22]: arr3 = np.array([True, False, True])  
      arr3.dtype
```

```
[22]: dtype('bool')
```

```
[23]: arr4 = np.array(["Hello", "World"])  
      arr4.dtype
```

```
[23]: dtype('<U5')      Tipo Unicode (U), little-endian (<) de 5 caracteres máximo
```

NumPy Básico

Crear un array de NumPy

```
[33]: arr1 = np.array([1,2,3,4], dtype=np.int8)
      print(arr1.dtype, arr1.nbytes, "bytes")

      int8 4 bytes
```

```
[34]: arr2 = np.array([1,2,3,4], dtype=np.int64)
      print(arr2.dtype, arr2.nbytes, "bytes")

      int64 32 bytes
```

```
[35]: arr3 = np.array([1, 3.14, True, "Hello"])
      arr3
```

```
[35]: array(['1', '3.14', 'True', 'Hello'], dtype='<U32')
```

- Se puede especificar el tipo al crear el array
- Si los tipos no son homogéneos se asigna el común

NumPy Básico

Crear un array de NumPy

Tipo de datos	Descripción
int8, int16, int32, int64	Enteros
uint8, uint16, uint32, uint64	Enteros sin signo
int_	Entero por defecto (según arquitectura de la máquina)
float16, float32, float64, float128	Coma flotante
float_	Coma flotante por defecto (según arquitectura de la máquina)
bool	Booleano
string_, unicode_	Strings

<https://numpy.org/doc/stable/reference/arrays.dtypes.html>

NumPy Básico

Crear un array de NumPy

```
[37]: np.linspace(1,10,5)
```

```
[37]: array([ 1. ,  3.25,  5.5 ,  7.75, 10.  ])
```

```
[38]: np.linspace(-10,10,20)
```

```
[38]: array([-10.          , -8.94736842, -7.89473684, -6.84210526,  
          -5.78947368, -4.73684211, -3.68421053, -2.63157895,  
          -1.57894737, -0.52631579,  0.52631579,  1.57894737,  
           2.63157895,  3.68421053,  4.73684211,  5.78947368,  
           6.84210526,  7.89473684,  8.94736842, 10.          ])
```

- Se puede crear un array en un espacio lineal obteniendo números uniformemente espaciados en un intervalo especificado.
- Se define el inicio y fin del intervalo y el número de elementos.
- NumPy calculará automáticamente el espaciado entre elementos

NumPy Básico

Crear un array de NumPy

```
[37]: np.linspace(1,10,5)
```

```
[37]: array([ 1. ,  3.25,  5.5 ,  7.75, 10.  ])
```

```
[38]: np.linspace(-10,10,20)
```

```
[38]: array([-10.          , -8.94736842, -7.89473684, -6.84210526,  
          -5.78947368, -4.73684211, -3.68421053, -2.63157895,  
          -1.57894737, -0.52631579,  0.52631579,  1.57894737,  
           2.63157895,  3.68421053,  4.73684211,  5.78947368,  
           6.84210526,  7.89473684,  8.94736842, 10.          ])
```

- Se puede crear un array en un espacio lineal obteniendo números uniformemente espaciados en un intervalo especificado.
- Se define el inicio y fin del intervalo y el número de elementos.
- NumPy calculará automáticamente el espaciado entre elementos

NumPy Básico

Crear un array de NumPy

```
[74]: np.random.randn(2,3)
```

```
[74]: array([[ -0.35782445, -1.13387621, -1.88517336],  
           [ 1.458706   , -0.46130698,  0.15408571]])
```

```
[76]: np.random.randint(1, 100, (2,3))
```

```
[76]: array([[70, 17, 79],  
           [44, 31, 48]])
```

- Generación de arrays aleatorios
- `randn` genera números siguiendo la distribución normal estándar (media (μ) = 0 y desviación estándar (σ) = 1) con la forma `shape`
- `randint` devuelve números enteros entre `start` y `end` y con la forma `shape`

NumPy Básico

Arrays multidimensionales

```
[61]: m = np.array([[1,2,3],[4,5,6],[7,8,9]])  
m
```

```
[61]: array([[1, 2, 3],  
           [4, 5, 6],  
           [7, 8, 9]])
```

```
[62]: m.ndim
```

```
[62]: 2
```

```
[63]: m.shape
```

```
[63]: (3, 3)
```

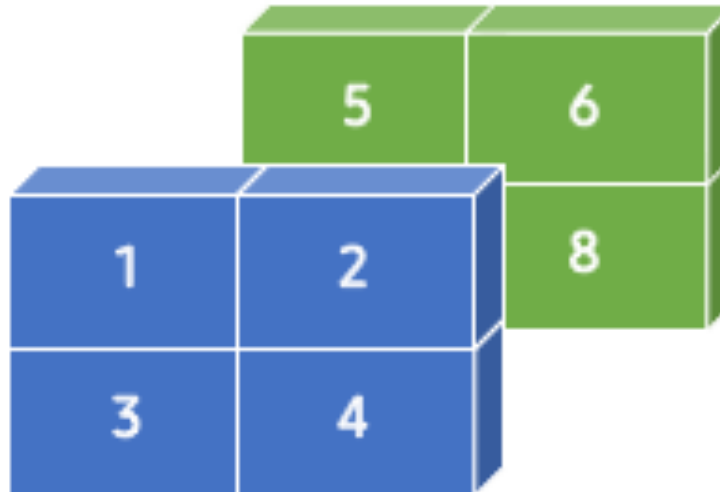
- Un array multidimensional se refiere a arrays de dos o más dimensiones
- Se pueden construir a partir de listas anidadas
- Las listas deben tener un tamaño compatible

NumPy Básico

Arrays multidimensionales

```
[64]: np.array([[[1,2],[3,4]],[[5,6],[7,8]]])
```

```
[64]: array([[[1, 2],  
             [3, 4]],  
           [[5, 6],  
           [7, 8]]])
```



- Ejemplo de array de 3 dimensiones

NumPy Básico

Arrays multidimensionales

```
[65]: np.zeros((2,3))
```

```
[65]: array([[0., 0., 0.],  
            [0., 0., 0.]])
```

```
[66]: np.ones((2,3))
```

```
[66]: array([[1., 1., 1.],  
            [1., 1., 1.]])
```

- Las funciones de creación de arrays admiten tuplas con la forma (shape) del array multi-dimensional

NumPy Básico

Arrays multidimensionales

```
[69]: np.zeros((2,3)).astype(np.int8)
```

```
[69]: array([[0, 0, 0],  
           [0, 0, 0]], dtype=int8)
```

```
[70]: np.zeros((2,3)).astype(np.float64)
```

```
[70]: array([[0., 0., 0.],  
           [0., 0., 0.]])
```

- El método `astype` permite cambiar el tipo de datos de un numpy array

NumPy Básico

Reshape

```
[10]: a = np.arange(15)
```

```
[11]: a
```

```
[11]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14])
```

```
[12]: a.reshape((3,5))
```

```
[12]: array([[ 0,  1,  2,  3,  4],  
           [ 5,  6,  7,  8,  9],  
           [10, 11, 12, 13, 14]])
```

```
[13]: a.reshape((3, -1))
```

```
[13]: array([[ 0,  1,  2,  3,  4],  
           [ 5,  6,  7,  8,  9],  
           [10, 11, 12, 13, 14]])
```

```
[16]: a.reshape((-1, 5))
```

```
[16]: array([[ 0,  1,  2,  3,  4],  
           [ 5,  6,  7,  8,  9],  
           [10, 11, 12, 13, 14]])
```

- Se puede cambiar el shape de un array con reshape
- Es muy óptimo ya que no cambia la representación interna del array
- El -1 en el shape actúa de comodín

NumPy Básico

Comparando listas con NumPy arrays

```
[78]: %time for i in range(1000): list(range(1_000_000))*2
```

```
CPU times: user 16.8 s, sys: 3.91 s, total: 20.7 s  
Wall time: 20.7 s
```

```
[79]: %time for i in range(1000): np.arange(1_000_000)*2
```

```
CPU times: user 338 ms, sys: 6.79 ms, total: 345 ms  
Wall time: 345 ms
```

- Python ejecuta la operación en un bucle interpretado (en C pero gestionado elemento a elemento).
- El array está en memoria contigua → acceso mucho más eficiente.
- Resultado: unas 60 veces más rápido que la lista.

NumPy Básico

Añadir elementos

```
[80]: a = np.array([1,2,3])
```

```
[81]: b = np.append(a, [4,5,6])
```

```
[82]: b
```

```
[82]: array([1, 2, 3, 4, 5, 6])
```

```
[83]: id(a), id(b)
```

```
[83]: (4677464208, 4677851184)
```

- Se puede extender un array con `np.append`
- Se crea un array nuevo

NumPy Básico

Añadir elementos

```
[84]: a = np.array([[1,2],[3,4]])  
a
```

```
[84]: array([[1, 2],  
           [3, 4]])
```

```
[85]: np.append(a, [[5,6]], axis=0)
```

```
[85]: array([[1, 2],  
           [3, 4],  
           [5, 6]])
```

- `axis=0` añade una nueva fila

NumPy Básico

Añadir elementos

```
[86]: a = np.array([[1,2],[3,4]])  
a
```

```
[86]: array([[1, 2],  
           [3, 4]])
```

```
[87]: np.append(a, [[9],  
                  [9]], axis=1)
```

```
[87]: array([[1, 2, 9],  
           [3, 4, 9]])
```

- `axis=1` añade una nueva columna

NumPy Básico

Eliminar elementos

```
[88]: a = np.array([[1,2,3],[4,5,6]])  
a
```

```
[88]: array([[1, 2, 3],  
          [4, 5, 6]])
```

```
[89]: np.delete(a, 0)
```

```
[89]: array([2, 3, 4, 5, 6])
```

```
[90]: a
```

```
[90]: array([[1, 2, 3],  
          [4, 5, 6]])
```

- Al eliminar un elemento se adapta el shape
- No se modifica el array original, se devuelve uno nuevo

NumPy Básico

Eliminar elementos

```
[88]: a = np.array([[1,2,3],[4,5,6]])  
a
```

```
[88]: array([[1, 2, 3],  
          [4, 5, 6]])
```

```
[91]: np.delete(a, 0, axis=0)
```

```
[91]: array([[4, 5, 6]])
```

```
[92]: np.delete(a, 1, axis=1)
```

```
[92]: array([[1, 3],  
          [4, 6]])
```

- Se puede eliminar una fila o una columna con el parámetro `axis`

Operaciones Básicas

Operadores suma, resta, multiplicación y división

```
[94]: np.array([1,2,3]) + np.array([4,5,6])
```

```
[94]: array([5, 7, 9])
```

```
[95]: np.array([1,2,3]) - np.array([4,5,6])
```

```
[95]: array([-3, -3, -3])
```

```
[96]: np.array([1,2,3]) * np.array([4,5,6])
```

```
[96]: array([ 4, 10, 18])
```

```
[97]: np.array([1,2,3]) / np.array([4,5,6])
```

```
[97]: array([0.25, 0.4 , 0.5 ])
```

- Todos estos operadores son elemento a elemento
- El shape debe coincidir

Operaciones Básicas

Comparando listas con arrays

```
[93]: [1,2,3] + [4,5,6]
```

```
[93]: [1, 2, 3, 4, 5, 6]
```

```
[94]: np.array([1,2,3]) + np.array([4,5,6])
```

```
[94]: array([5, 7, 9])
```

- Operador suma en listas → concatena las listas
- Operador suma en NumPy arrays → suma elemento a elemento

Operaciones Básicas

repeat y tile

```
[98]: np.repeat(np.array([1,2,3]), 3)
```

```
[98]: array([1, 1, 1, 2, 2, 2, 3, 3, 3])
```

```
[99]: np.tile(np.array([1,2,3]), 3)
```

```
[99]: array([1, 2, 3, 1, 2, 3, 1, 2, 3])
```

```
100]: a = np.array([[0,1,2],  
                   [3,4,5],  
                   [6,7,8]])  
      np.tile(a, 3)
```

```
100]: array([[0, 1, 2, 0, 1, 2, 0, 1, 2],  
            [3, 4, 5, 3, 4, 5, 3, 4, 5],  
            [6, 7, 8, 6, 7, 8, 6, 7, 8]])
```

- repeat repite cada elemento
- tile repite el array entero

Operaciones Básicas

Operaciones lógicas

```
[2]: arr = np.array([10,20,30,40])
```

```
[3]: arr > 10
```

```
[3]: array([False,  True,  True,  True])
```

```
[4]: arr[arr>10]
```

```
[4]: array([20, 30, 40])
```

- Las operaciones lógicas se comparan elemento a elemento
- Devuelven un array de booleanos
- Ese array puede usarse como máscara

Operaciones Básicas

Funciones matemáticas, trigonométricas y estadísticas

```
[5]: arr = np.array([1,2,3,4])
```

```
[6]: arr ** 2
```

```
[6]: array([ 1,  4,  9, 16])
```

```
[7]: np.sqrt(arr)
```

```
[7]: array([1.          , 1.41421356, 1.73205081, 2.          ])
```

```
[8]: np.sum(arr)
```

```
[8]: 10
```

```
[9]: np.mean(arr)
```

```
[9]: 2.5
```

Operaciones Básicas

Funciones matemáticas, trigonométricas y estadísticas

Función	Descripción
sin, cos, tan	Funciones trigonométricas
arcsin, arccos, arctan	Funciones trigonométricas inversas
round	Redondea a un número de decimales válido
floor, ceil	Redondea al entero más cercano hacia arriba/abajo
fix	Redondea hacia el 0, sin importar el signo
prod	Devuelve el producto de todos los elementos de un array
cumsum	Devuelve la suma acumulativa de un array
exp, log	Funciones exponenciales y logarítmicas
unique	Devuelve los valores únicos de un array
min, max, argmin, argmax	Mínimo, máximo y los índices correspondientes
sum, mean, var, std, median	Funciones estadísticas

Operaciones Básicas

Funciones matemáticas, trigonométricas y estadísticas

```
[17]: a = np.array([[1,2,3],[4,5,6]])
```

```
[18]: a.mean()
```

```
[18]: 3.5
```

```
[19]: a.mean(axis=0)
```

```
[19]: array([2.5, 3.5, 4.5])
```

```
[20]: a.mean(axis=1)
```

```
[20]: array([2., 5.])
```

- Las funciones estadísticas también pueden calcularse por fila o por columna

Operaciones Básicas

Operaciones matriciales: producto escalar y producto matricial

```
[22]: a = np.array([1, 2, 3])  
      b = np.array([4, 5, 6])  
      np.dot(a, b)    # → 32  (1*4 + 2*5 + 3*6)
```

```
[22]: 32
```

```
[23]: A = np.array([[1, 2], [3, 4]])  
      B = np.array([[5, 6], [7, 8]])  
      np.dot(A, B)
```

```
[23]: array([[19, 22],  
            [43, 50]])
```

- En vectores 1D `np.dot` hace el producto escalar
- Con matrices 2D hace la multiplicación matricial

Operaciones Básicas

Operaciones matriciales: producto escalar y producto matricial

```
[24]: A = np.array([[1, 2], [3, 4]])  
      B = np.array([[5, 6], [7, 8]])  
      np.matmul(A, B)
```

```
[24]: array([[19, 22],  
            [43, 50]])
```

```
[25]: a = np.array([1, 2])  
      B = np.array([[3, 4], [5, 6]])  
      np.matmul(a, B) # → [13 16] (vector resultado)
```

```
[25]: array([13, 16])
```

```
[33]: A @ B
```

```
[33]: array([[13, 16],  
            [29, 36]])
```

- `np.matmul` solo está definido para arrays $\geq 2D$. Para 2D, es igual que `np.dot`
- Para 1D, no devuelve un escalar sino que promueve a 2D para hacer multiplicación de matriz con vector
- El operador `@` se introdujo en Python 3.5 específicamente como operador de multiplicación matricial. Es equivalente a `np.matmul`

Operaciones Básicas

Operaciones matriciales: traspuesta

```
[35]: A
```

```
[35]: array([[1, 2],  
           [3, 4]])
```

```
[36]: A.transpose()
```

```
[36]: array([[1, 3],  
           [2, 4]])
```

```
[37]: A.T
```

```
[37]: array([[1, 3],  
           [2, 4]])
```

- La traspuesta de un array se calcula con `transpose()` o con `np.transpose()`
- Existe un atajo para recordar a la notación matemática A^T que es `A.T`

Operaciones Básicas

Operaciones matriciales: inversa y determinante

```
[38]: np.linalg.inv(A)
```

```
[38]: array([[ -2. ,  1. ],  
           [ 1.5, -0.5]])
```

```
[39]: np.linalg.det(A)
```

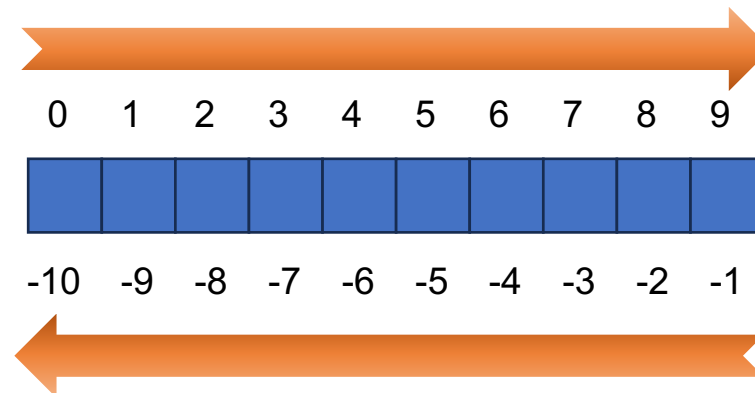
```
[39]: -2.0000000000000004
```

- `np.linalg.inv` usa LAPACK con factorización LU. $A=PLU$ (permutación, triangular inferior, triangular superior)
- No es exacta, siempre es una aproximación numérica.
- `np.linalg.det` usa LU decomposition (vía LAPACK).
- El determinante no es una medida numéricamente estable para muchas aplicaciones.
- Es un indicador de invertibilidad : $\det(A) \neq 0 \rightarrow$ la matriz tiene inversa (es “buena”);
 $\det(A) = 0 \rightarrow$ la matriz es singular (no se puede invertir).

Indexing y Slicing

Introducción

- Para seleccionar un elemento usando su posición usamos el **indexing**.
- Para seleccionar un subconjunto de elementos usamos el **slicing**.
- En arrays unidimensionales se selecciona un elemento usando su **posición**
- Se empieza a contar en **0** (igual que en las listas).
- Podemos usar **indexing negativo** para empezar por el último elemento.



Indexing y Slicing

Indexing

```
[40]: a = np.array([0,1,2,3])  
      b = np.array([[0,1,2,3],[4,5,6,7],[8,9,10,11]])
```

```
[41]: a[0]
```

```
[41]: 0
```

```
[42]: b[0]
```

```
[42]: array([0, 1, 2, 3])
```

```
[43]: b[0][1]
```

```
[43]: 1
```

- El indexing se hace igual que el de las listas de Python.
- El arrays multi-dimensionales se puede acceder como con listas anidadas.

Indexing y Slicing

Indexing

```
[59]: b[0,1]
```

```
[59]: 1
```

```
[60]: b[0]
```

```
[60]: array([0, 1, 2, 3])
```

- A diferencia de las listas de Python, el acceso multi-dimensional se puede realizar con las dimensiones separadas por **comas**.
- Es una forma más **cómoda** de acceder a los valores de una matriz n-dimensional.
- Si el array es multi-dimensional y solo se introduce una dimensión se devuelve la **fila entera** seleccionada.

Indexing y Slicing

Indexing

```
[55]: a[[1,3,4]]
```

```
[55]: array([ 20, 100,  50])
```

```
[57]: a[[4,1,3]]
```

```
[57]: array([ 50,  20, 100])
```

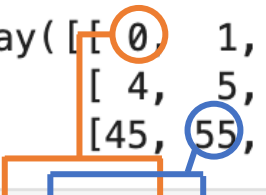
- El operador de indexing (`[]`) permite seleccionar varios elementos a la vez.
- Los elementos NO tienen porqué ser consecutivos.
- Es necesario pasar una lista con los índices al operador (de ahí los dobles corchetes).

Indexing y Slicing

Indexing

```
[71]: b
```

```
[71]: array([[ 0,  1,  2,  3],  
          [ 4,  5,  6,  7],  
          [45, 55, 65, 75]])
```



```
[72]: b[[0,2], [0,1]]
```

```
[72]: array([ 0, 55])
```

- El indexado múltiple funciona igual en varias dimensiones.
- Primero se introduce la lista de filas y luego la lista de columnas.
- Cuidado! NO es una lista de parejas fila-columna

Indexing y Slicing

Indexing

```
[45]: a = np.array([10,20,30,40,50])
```

```
[46]: a[3]
```

```
[46]: 40
```

```
[47]: a[3] = 100
```

```
[49]: a[3]
```

```
[49]: 100
```

- Es posible cambiar un valor de un array mediante el operador de asignación y el indexing.
- Únicamente podemos asignar valores del mismo tipo, sino NumPy tratará de convertirlo o lanzará un error `ValueError`.

Indexing y Slicing

Indexing

```
[66]: b[2,2] = 999  
b
```

```
[66]: array([[ 0,  1,  2,  3],  
          [ 4,  5,  6,  7],  
          [ 8,  9, 999, 11]])
```

```
[68]: b[2] = np.array([45, 55, 65, 75])
```

```
[69]: b
```

```
[69]: array([[ 0,  1,  2,  3],  
          [ 4,  5,  6,  7],  
          [45, 55, 65, 75]])
```

- Se puede asignar un nuevo valor al array usando este indexing.
- Si únicamente se selecciona la primera dimensión se sustituye la fila entera (ha de coincidir el shape).

Indexing y Slicing

Indexing

```
[21]: arr = np.empty((8,4))  
arr
```

```
[21]: array([[0., 0., 0., 0.],  
           [0., 0., 0., 0.],  
           [0., 0., 0., 0.],  
           [0., 0., 0., 0.],  
           [0., 0., 0., 0.],  
           [0., 0., 0., 0.],  
           [0., 0., 0., 0.],  
           [0., 0., 0., 0.]])
```

```
[22]: for i in range(8):  
      arr[i] = i
```

```
[23]: arr
```

```
[23]: array([[0., 0., 0., 0.],  
           [1., 1., 1., 1.],  
           [2., 2., 2., 2.],  
           [3., 3., 3., 3.],  
           [4., 4., 4., 4.],  
           [5., 5., 5., 5.],  
           [6., 6., 6., 6.],  
           [7., 7., 7., 7.]])
```

- Inicializamos un array vacío con `np.empty` y asignándole valores a cada fila

Indexing y Slicing

Indexing

```
[24]: arr[[4, 3, 0, 6]]
```

```
[24]: array([[4., 4., 4., 4.],  
          [3., 3., 3., 3.],  
          [0., 0., 0., 0.],  
          [6., 6., 6., 6.]])
```

```
[25]: arr[[-3, -5]]
```

```
[25]: array([[5., 5., 5., 5.],  
          [3., 3., 3., 3.]])
```

- Seleccionar filas en un orden específico
- Podemos usar índices negativos para seleccionar filas desde el final

Indexing y Slicing

Indexing

```
[3]: a = np.array([1,2,3,4,5,6])
```

```
[5]: a[a>3]
```

```
[5]: array([4, 5, 6])
```

```
[6]: a[a%2==0]
```

```
[6]: array([2, 4, 6])
```

- Se puede utilizar el operador de indexing con operaciones lógicas
- Funcionan como máscaras

Indexing y Slicing

Indexing

```
[17]: names = np.array(['Bob', 'Joe', 'Tom', 'Bob', 'Tom', 'Joe', 'Joe'])  
names
```

```
[17]: array(['Bob', 'Joe', 'Tom', 'Bob', 'Tom', 'Joe', 'Joe'], dtype='<U3')
```

```
[18]: names == 'Bob'
```

```
[18]: array([ True, False, False,  True, False, False, False])
```

```
[19]: names[names=='Bob']
```

```
[19]: array(['Bob', 'Bob'], dtype='<U3')
```

- El indexing con operaciones lógicas funciona también con otros tipos de datos
- Si la operación lógica se puede evaluar el indexing funcionará, puesto que devuelve un array de booleanos

Indexing y Slicing

Indexing

```
[26]: arr = np.arange(20)
```

```
[27]: arr_mask = arr%5==0
```

```
[28]: arr[arr_mask]
```

```
[28]: array([ 0,  5, 10, 15])
```

```
[29]: arr_mask2 = arr >= 10
```

```
[32]: arr[arr_mask & arr_mask2]
```

```
[32]: array([10, 15])
```

```
[33]: arr[arr_mask | arr_mask2]
```

```
[33]: array([ 0,  5, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19])
```

- Las máscaras se pueden combinar con operadores lógicos como el AND (&) y el OR (|)

Indexing y Slicing

Slicing

```
[7]: a[:3]
```

```
[7]: array([1, 2, 3])
```

```
[8]: a[2:]
```

```
[8]: array([3, 4, 5, 6])
```

- Con el slicing seleccionamos subconjuntos del array
- Funciona igual que las listas de Python

Indexing y Slicing

Slicing

```
[9]: arr2d = np.array([[1,2,3],[4,5,6],[7,8,9]])
```

```
[10]: arr2d
```

```
[10]: array([[1, 2, 3],  
           [4, 5, 6],  
           [7, 8, 9]])
```

```
[11]: arr2d[:2]
```

```
[11]: array([[1, 2, 3],  
           [4, 5, 6]])
```

- En arrays de 2 dimensiones seleccionaremos filas enteras si usamos una dimensión

Indexing y Slicing

Slicing

```
[13]: arr2d[:2, 1:]
```

```
[13]: array([[2, 3],  
          [5, 6]])
```

```
[14]: arr2d[1, :2]
```

```
[14]: array([4, 5])
```

```
[15]: arr2d[:2, 2]
```

```
[15]: array([3, 6])
```

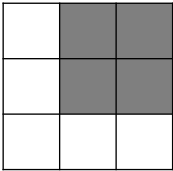
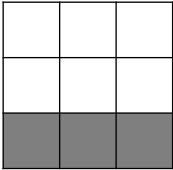
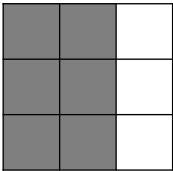
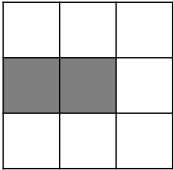
```
[16]: arr2d[:, :1]
```

```
[16]: array([[1],  
          [4],  
          [7]])
```

- En arrays de 2 dimensiones si usamos dos dimensiones podemos seleccionar submatrices
- Funciona en cualquier dimensión mayor de dos

Indexing y Slicing

Slicing

	Código	Forma del slice
	<code>arr[:2, 1:]</code>	(2,2)
	<code>arr[2]</code> <code>arr[2, :]</code> <code>arr[2:, :]</code>	(3,) (3,) (1,3)
	<code>arr[:, :2]</code>	(3,2)
	<code>arr[1, :2]</code> <code>arr[1:2, :2]</code>	(2,) (1,2)

Indexing y Slicing

Transposiciones

```
[34]: arr = np.arange(15).reshape((3,5))
```

```
[35]: arr
```

```
[35]: array([[ 0,  1,  2,  3,  4],  
          [ 5,  6,  7,  8,  9],  
          [10, 11, 12, 13, 14]])
```

```
[36]: arr.transpose()
```

```
[36]: array([[ 0,  5, 10],  
          [ 1,  6, 11],  
          [ 2,  7, 12],  
          [ 3,  8, 13],  
          [ 4,  9, 14]])
```

```
[37]: arr.T
```

```
[37]: array([[ 0,  5, 10],  
          [ 1,  6, 11],  
          [ 2,  7, 12],  
          [ 3,  8, 13],  
          [ 4,  9, 14]])
```

- Ya sabemos hacer la transpuesta de una matriz de dos dimensiones
- ¿Se puede hacer con más dimensiones?

Indexing y Slicing

Transposiciones

```
[39]: arr
```

```
[39]: array([[ 0,  1,  2,  3,  4],  
           [ 5,  6,  7,  8,  9],  
           [10, 11, 12, 13, 14]])
```

```
[38]: arr.transpose((1,0))
```

```
[38]: array([[ 0,  5, 10],  
           [ 1,  6, 11],  
           [ 2,  7, 12],  
           [ 3,  8, 13],  
           [ 4,  9, 14]])
```

- Al transponer podemos dar como argumento una tupla de ejes
- Esta tupla indica que eje se intercambia por otro
- En el ejemplo hay un 1 en la posición 0 de la tupla, indicando que el eje 0 se intercambia por el 1

Indexing y Slicing

Transposiciones

```
[42]: arr = np.arange(16).reshape((2,2,4))  
arr
```

```
[42]: array([[[ 0,  1,  2,  3],  
            [ 4,  5,  6,  7]],  
          [[ 8,  9, 10, 11],  
            [12, 13, 14, 15]])
```

```
[43]: arr.transpose((1,0,2))
```

```
[43]: array([[[ 0,  1,  2,  3],  
            [ 8,  9, 10, 11]],  
          [[ 4,  5,  6,  7],  
            [12, 13, 14, 15]])
```

```
[44]: arr.transpose((2,0,1))
```

```
[44]: array([[[ 0,  4],  
            [ 8, 12]],  
          [[ 1,  5],  
            [ 9, 13]],  
          [[ 2,  6],  
            [10, 14]],  
          [[ 3,  7],  
            [11, 15]])
```

- En tres o más dimensiones funciona del mismo modo

Indexing y Slicing

Transposiciones

```
[45]: arr.swapaxes(1,2)
```

```
[45]: array([[[ 0,  4],
              [ 1,  5],
              [ 2,  6],
              [ 3,  7]],

            [[ 8, 12],
              [ 9, 13],
              [10, 14],
              [11, 15]])
```

```
[46]: arr.swapaxes(0,1)
```

```
[46]: array([[[ 0,  1,  2,  3],
              [ 8,  9, 10, 11]],

            [[ 4,  5,  6,  7],
              [12, 13, 14, 15]])
```

- El método `swapaxes` funciona de forma similar a `transpose`.

Conclusiones

Qué hemos aprendido

- Los NumPy arrays son una estructura de datos ideal para vectores y matrices numéricos
- Implementa las operaciones aritméticas y de álgebra más habituales
- Permite minimizar el número de bucles gracias a operaciones vectorizadas
- Tiene un potente operador de indexado y slicing
- Tiene un rendimiento muy alto
- Es muy utilizado por la mayoría de módulos de Ciencia de Datos y de IA